ING SOFTWARE I

Jisbaj Gamarra Salas
jisbaj.gamarra@unsaac.edu.pe

Departamento Académico de Ing. Informática



GESTIÓN DE LA CONFIGURACIÓN Y CONTROL DE CAMBIO

Propósito de la sesión

Identificar

Qué se controla y cuál es la versión acordada.

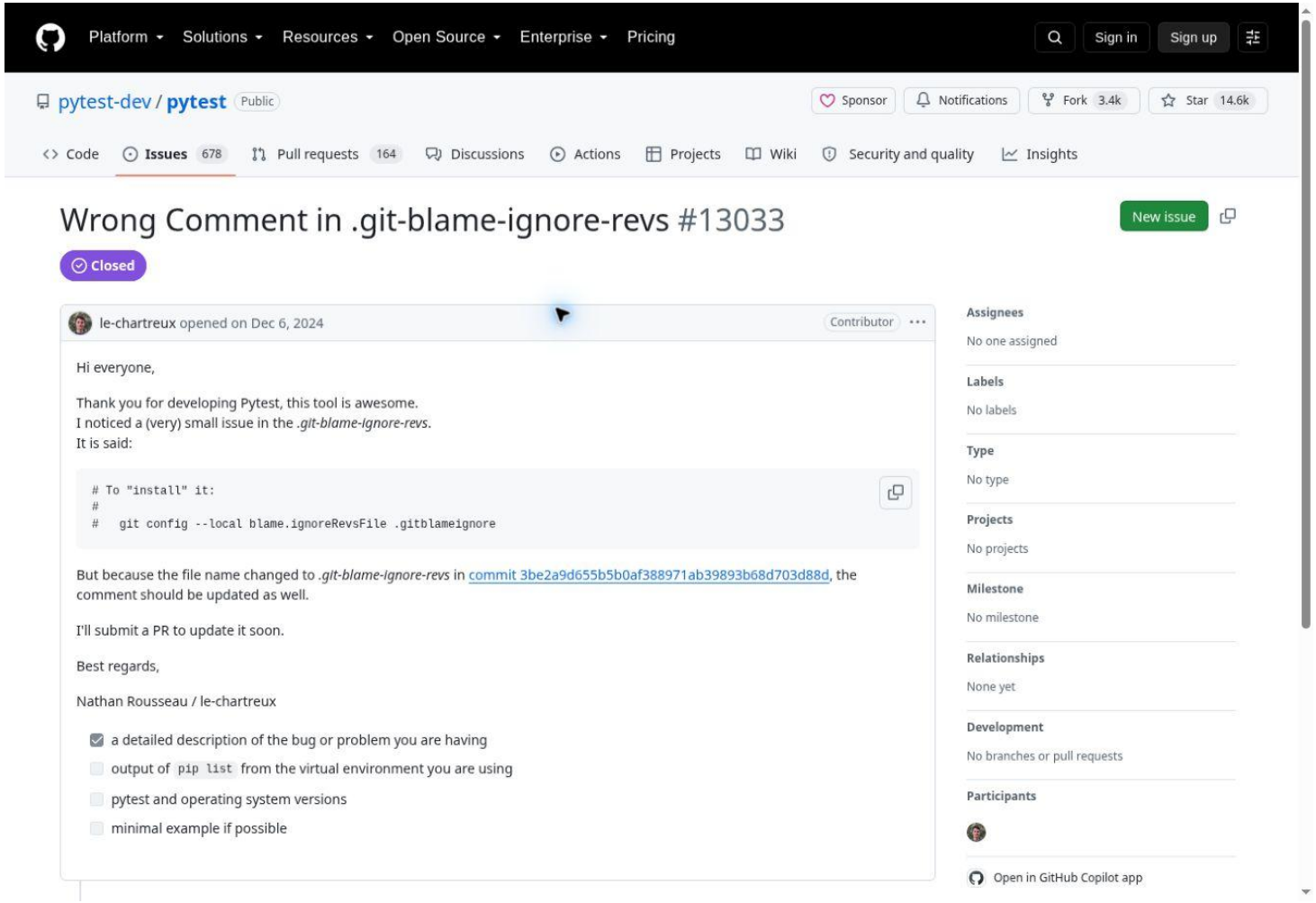
Colaborar

Cómo proponer, revisar e integrar un cambio.

Demostrar

Un recorrido con evidencia desde la tarea hasta «Hecho».

El caso real: una instrucción quedó atrás



https://github.com/pytest-dev/pytest/issues/13033

pytest · #13033

Cambió el nombre del archivo.

La instrucción seguía usando el anterior.

¿Basta con guardar el código?

¿Qué necesitamos conservar para reconstruir una entrega?

Identificar versiones, acordar una referencia y controlar sus cambios.

Ítems de configuración: más que código

Un ítem tiene identidad, versión, ubicación y responsable.



src/

tests/

Lockfile (ej. Package-lock.json)

docs/

También: requisitos, datos de prueba, scripts y configuración de automatización.

Baseline: una referencia acordada

LÍNEA BASE

Un conjunto identificado y aprobado.

Identidad

Versión + commit de referencia

Contenido

Código, pruebas y documentos

Reproducción

Dependencias e instrucciones

Aprobación

Quién, cuándo y con qué evidencia

Una etiqueta de Git puede señalarla; el acuerdo la convierte en línea base.

Control de cambios: decidir antes de integrar

Una solicitud deja de ser un mensaje perdido y se vuelve una decisión.



Impacto: comportamiento, interfaces, pruebas, documentación y entrega.

Si se rechaza o pospone, también se conserva la razón.

Trazabilidad: seguir el porqué del cambio

Los enlaces permiten ir de la necesidad a la implementación y volver.

Issue #13033 → **Pull Request #13034** → **Commit 0526346** → **Integrado 9fc8a3e**

En el caso real: reporte → corrección → revisión → integración.

En su proyecto, añadan la prueba y la versión que contienen el cambio.

pytest · PR #13034, 2024 / pytest · Issue #13033, 2024

Ejercicio · Diseñar el inventario y la línea base

¿Podría otro equipo reconstruir su próxima demostración?

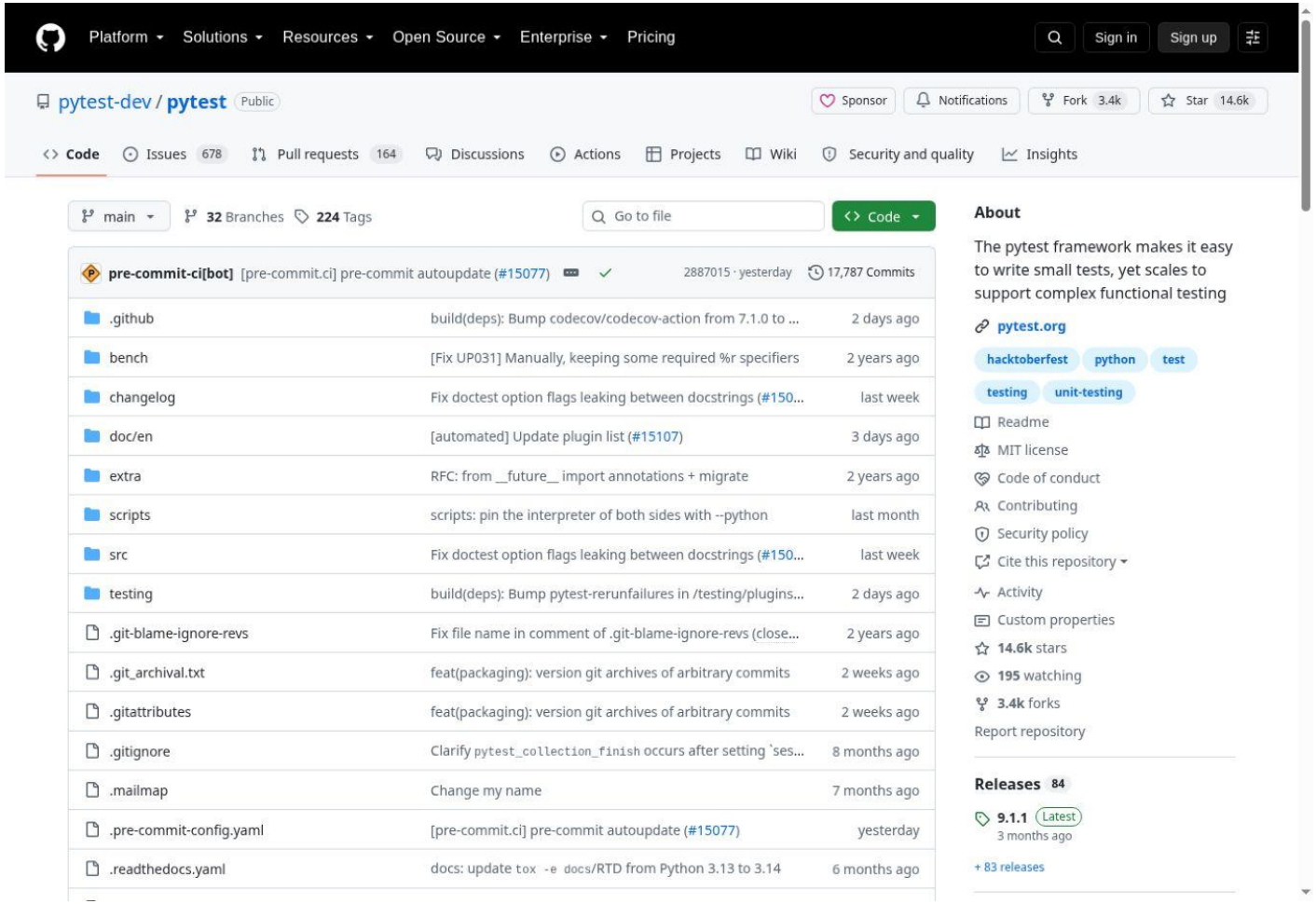
1. Elijan 4 ítems y anoten ruta, versión y responsable.
2. Definan qué evidencia permitirá aprobar la línea base.
3. Acuerden quién autoriza un cambio que afecte requisitos.

Evidencia: docs/configuracion.md, con inventario y regla de aprobación.

Ahora sabemos qué conservar. ¿Cómo cambiamos sin perder la historia?

Repositorio, commit, branch, merge y tag.

Repositorio: archivos, historia y referencias



Git

Registra versiones y sus relaciones.

GitHub

Aloja el remoto, issues y revisiones.

Local ↔ remoto
fetch / push

pytest-dev/pytest · Captura real / Pro Git

Commit: una instantánea con contexto

Directorio de trabajo	→	Preparación (staging)	→	Repositorio local	→	Repositorio remoto
editar		**git add**		**git commit**		**git push**

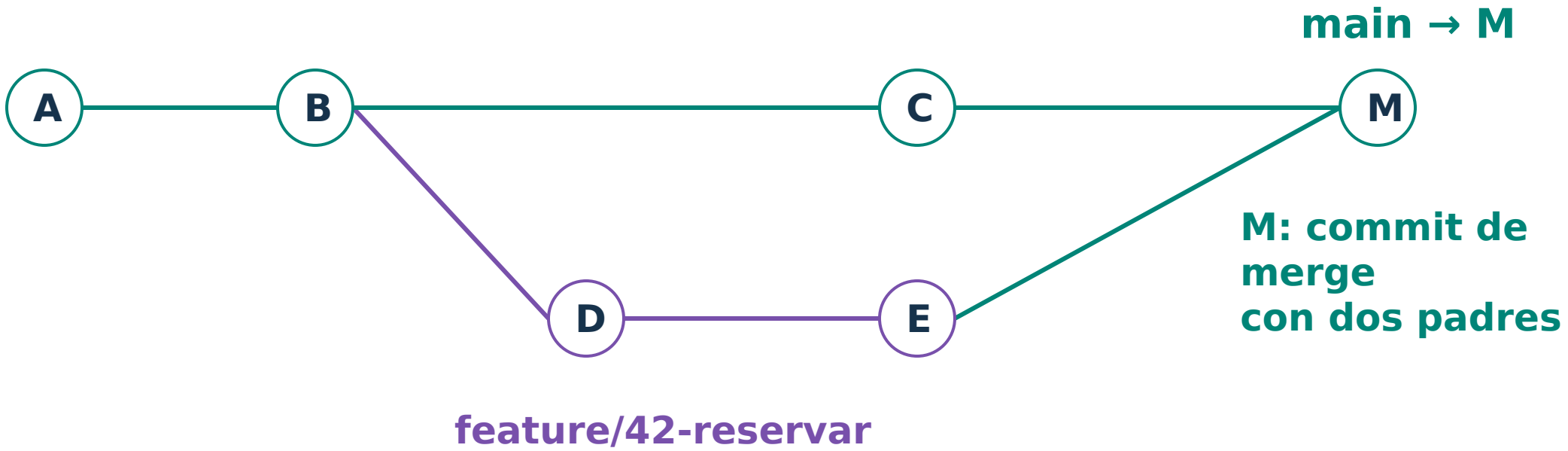
Un commit coherente — Una intención, mensaje claro y contenido revisado.

Antes de confirmar — git status → git diff --staged

Guardar un archivo, crear un commit y hacer push son acciones distintas.

Branch y merge: trabajar y volver a unir

Una rama es una referencia móvil; merge integra historias.



La rama aísla el trabajo; la revisión decide cuándo puede integrarse.

Merge: cuando Git necesita una decisión humana

Rama A

Límite de reserva: 2 horas.
La regla exige ampliar el cupo.

Rama B

Límite de reserva: 1 hora.
La regla exige repartir turnos.

Leer ambas intenciones → **Acordar la regla** → **Editar y comprobar** → **Confirmar la resolución**

Resolver el texto no demuestra que el comportamiento sea correcto.

Tag, release y baseline: tres propósitos

Tag · etiqueta

Localizar un punto en la historia.

v0.1.0 → commit

Release · entrega

Publicar notas y artefactos asociados a una versión.

Baseline · referencia

Acordar qué conjunto se considera aprobado y cómo cambiarlo.

https://github.com/git/git/tags

Aplicación al equipo: etiqueta + manifiesto de versiones + aprobación.

Línea base en software

Cómo se relaciona con los commit y los tag

¿Qué es una línea base?

Es una versión aprobada y estable del producto o de un conjunto de artefactos. Sirve como punto de referencia para controlar cambios.

Ejemplos de artefactos:

- **Código fuente** — Archivos de código del proyecto.
- **Documentación** — Guías, manuales, requisitos, etc.
- **Pruebas** — Casos de prueba, scripts, resultados.
- **Configuración** — Archivos de configuración, entornos, dependencias.

Desde aquí los cambios se comparan, controlan y trazan.

Cómo surgen los commit



Cada **commit** guarda un cambio pequeño y rastreable en el historial.

Cómo aparece el tag

Cuando un conjunto de commits representa una **versión** estable, se identifica con un **tag**.

Repositorio con tags



El **tag** nombra una versión específica del historial.

En resumen

- **Commit** — Registra un cambio puntual.
- **Tag** — Marca una versión importante.
- **Línea base** — Es la versión aprobada que se toma como referencia.

Muchos commits construyen una versión; cuando esa versión se aprueba, puede convertirse en línea base y marcarse con un tag.

git / git Public

Notifications | Fork 28.5k

Code | Pull requests 396 | Actions | Security and quality 32 | Insights

Releases | Tags

Tags

v2.56.0 ...

Verified

3 days ago · a018953 · zip · tar.gz

v2.56.0-rc2 ...

Verified

last week · 3bc0341 · zip · tar.gz

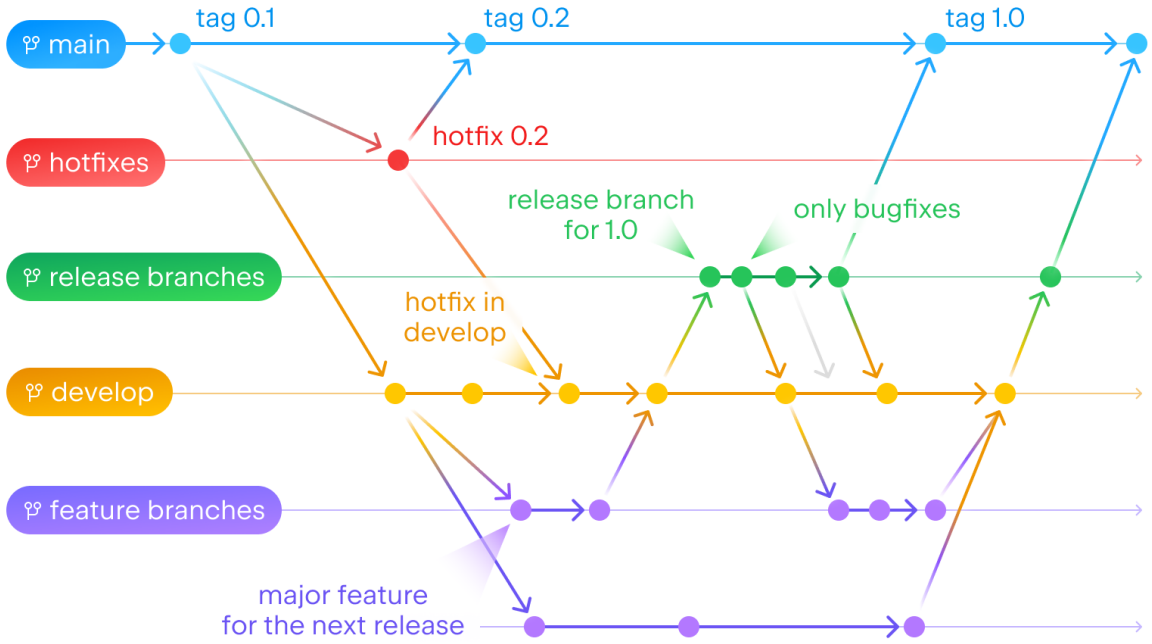
v2.56.0-rc1 ...

Verified

2 weeks ago · 12cb629 · zip · tar.gz

Estrategias de ramificación

Git flow



- La rama **main** es solo para código en producción.
- La rama **develop** es para código en desarrollo.
- Las ramas **feature** se crean a partir de la rama **develop**.
- Las ramas **hotfix** se crean a partir de la rama **main**.
- Las ramas **release** se crean a partir de la rama **develop**.

https://blog.jetbrains.com/space/2023/04/18/space-git-flow/#github-flow

Flujo de GitHub



- La rama `main` es solo para código en producción.
- El desarrollo se realiza en ramas `feature` independientes, que luego se fusionan en la rama `main` cuando están listas.
- Los Pull Requests se utilizan para revisiones de código y deben cumplir las reglas de protección de la rama antes de ser fusionados.
- Las versiones (releases) se crean a partir de la rama `main` y se etiquetan para facilitar el control de versiones.

Ejercicio · Un primer cambio en una rama

Creen una versión identificable del acuerdo de trabajo.

1. Elijan una estrategia de ramificación. Clonen o abran su repositorio y creen la rama docs/entorno.
2. Editen README, revisen el diff y hagan un commit.
3. Publiquen la rama y localicen el commit en el remoto.

Evidencia: URL del commit y nombre de la rama

Un cambio está listo.
¿Quién debe entenderlo antes de unirlo?

Issues, Pull Requests, revisión y mensajes con intención.

Issues: convertir una necesidad en trabajo

«No funciona»



No permite reproducir
ni comprobar una solución.

«Evitar reservas duplicadas»

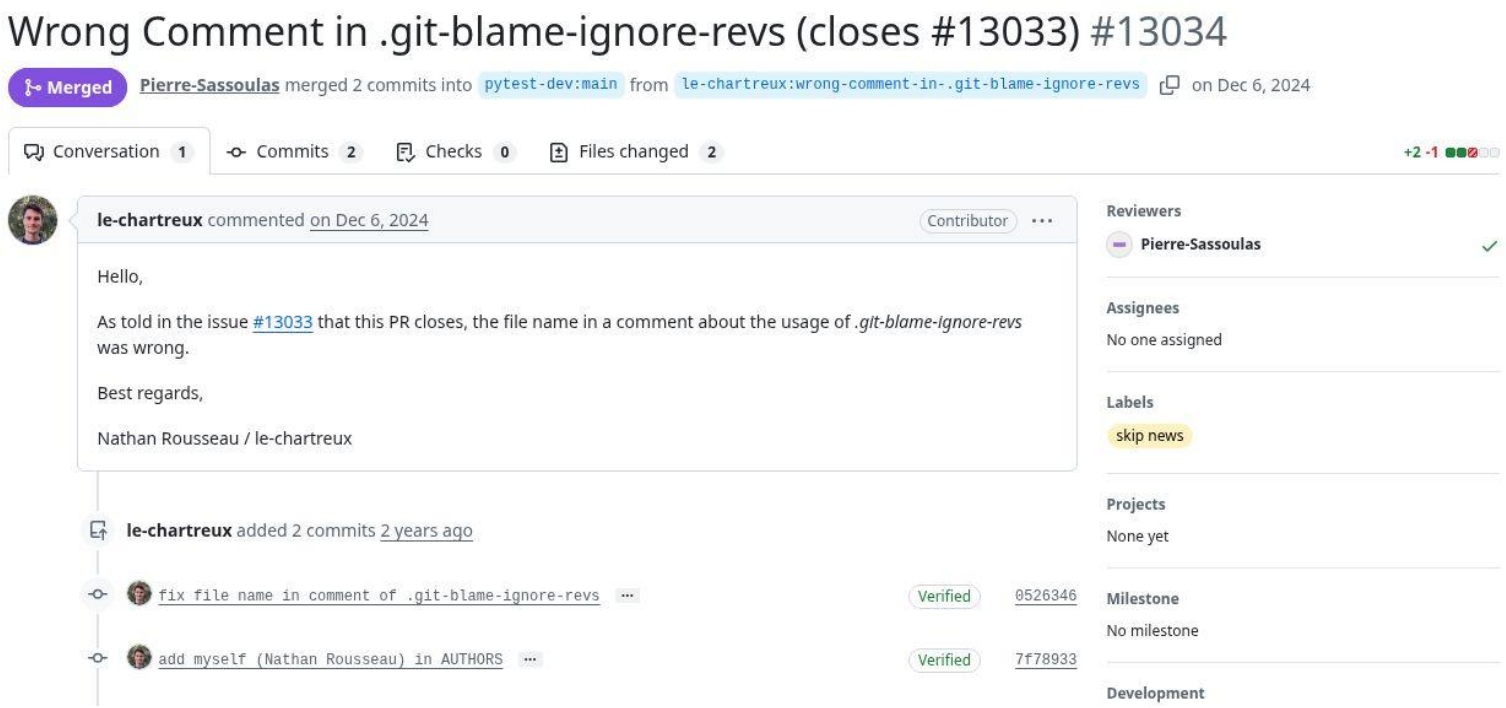


- Contexto y pasos para reproducir.
- Resultado actual y esperado.
- Criterios de aceptación.
- Responsable y prioridad.

Un issue responde: ¿qué problema existe y cómo sabremos que quedó resuelto?

GitHub Docs · GitHub flow / pytest · Issue #13033, 2024

Pull Request: una propuesta para integrar



pytest · #13034

- Propone unir una rama.
- Explica el cambio.
- Invita a revisarlo.

El PR conserva la conversación.

pytest · PR #13034, 2024 / GitHub Docs · GitHub flow

Flujo de Proceso de un Pull Request

Cómo un cambio pasa de una rama local a la rama principal mediante revisión, pruebas y fusión

1 Creación de la rama



El desarrollador crea una rama local, por ejemplo **"feature/nueva-funcion"**, a partir de **"main"** o **"develop"** para trabajar en una tarea específica.

2 Desarrollo y commits



Escribe el código en su rama local y realiza commits con mensajes claros a medida que avanza.

3 Publicación de la rama



Hace *push* de la rama local al repositorio remoto en GitHub, GitLab o Bitbucket.

4 Apertura del Pull Request



- Se solicita fusionar "feature/nueva-funcion" hacia "main".
- Se completa la plantilla: qué se hizo, por qué se hizo y cómo probarlo.
- Se asignan revisores.

5 Pruebas automáticas (CI/CD)



La integración continua ejecuta pruebas unitarias, linters y análisis de seguridad sobre el PR.

Si hay observaciones, se corrigen los cambios y se vuelve a subir (retroalimentación)

6 Revisión de código



- Los revisores analizan el código y dejan comentarios.
- Si hay observaciones, el desarrollador corrige y vuelve a subir cambios.
- El PR se actualiza automáticamente.

7 Aprobación



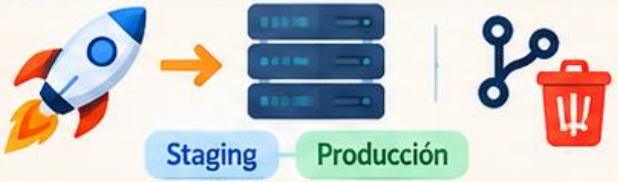
Cuando el código cumple los estándares y pasa las pruebas, los revisores aprueban el PR.

8 Fusión (Merge)



El autor o un encargado fusiona el Pull Request. La rama "feature/nueva-funcion" se integra en "main".

9 Despliegue y cierre



El código de la rama principal se despliega en Staging o Producción y la rama de la función se elimina para mantener limpio el repositorio.

Conceptos clave en un Pull Request

Rama
Una rama es unaa línea de desarrollo paralela para trabajar en una tarea específica.

Commit
Un commit guarda un cambio pequeño y rastreable en el historial.

Review
La revisión de código mejora la calidad, detecta errores y comparte conocimiento.

Merge
La fusión integra los cambios aprobados en la rama principal (por ejemplo, main).

Idea clave: un Pull Request no solo integra código; también documenta cambios, activa pruebas y facilita la colaboración del equipo.

Code Review: mirar el diff y pedir evidencia

2 .git-blame-ignore-revs

```
@@ -5,7 +5,7 @@
5  5   #
6  6   # To "install" it:
7  7   #
8     - #   git config --local blame.ignoreRevsFile .gitblameignore
   8  + #   git config --local blame.ignoreRevsFile .git-blame-ignore-revs
9  9
10 10  # run black
11 11  703e4b11ba76171eccd3f13e723c47b810ded7ef
```

Correctitud

¿La instrucción usa el archivo correcto?

Impacto

¿Queda otra referencia desactualizada?

Evidencia

¿Cómo comprobó el autor que ahora funciona?

pytest · PR #13034, archivos modificados

Convenciones de commits: contar la intención

tipo(ámbito): descripción

docs(entorno): explica cómo ejecutar las pruebas

«cambios» → no comunica intención
«fix(reservas): evita aceptar horarios ocupados» → sí la comunica

Acuerden idioma, tipos y tamaño de cambio. El mensaje debe reflejar el diff.

Conventional Commits 1.0.0

Ejercicio · Revisar el cambio de otra persona

Abran un Pull Request desde la rama del ejercicio anterior.

1. Creen un issue y enlácenlo al PR: problema, cambio y verificación.
2. Otra persona revise el diff y haga un comentario concreto.
3. Corrijan o respondan; registren la decisión de revisión.

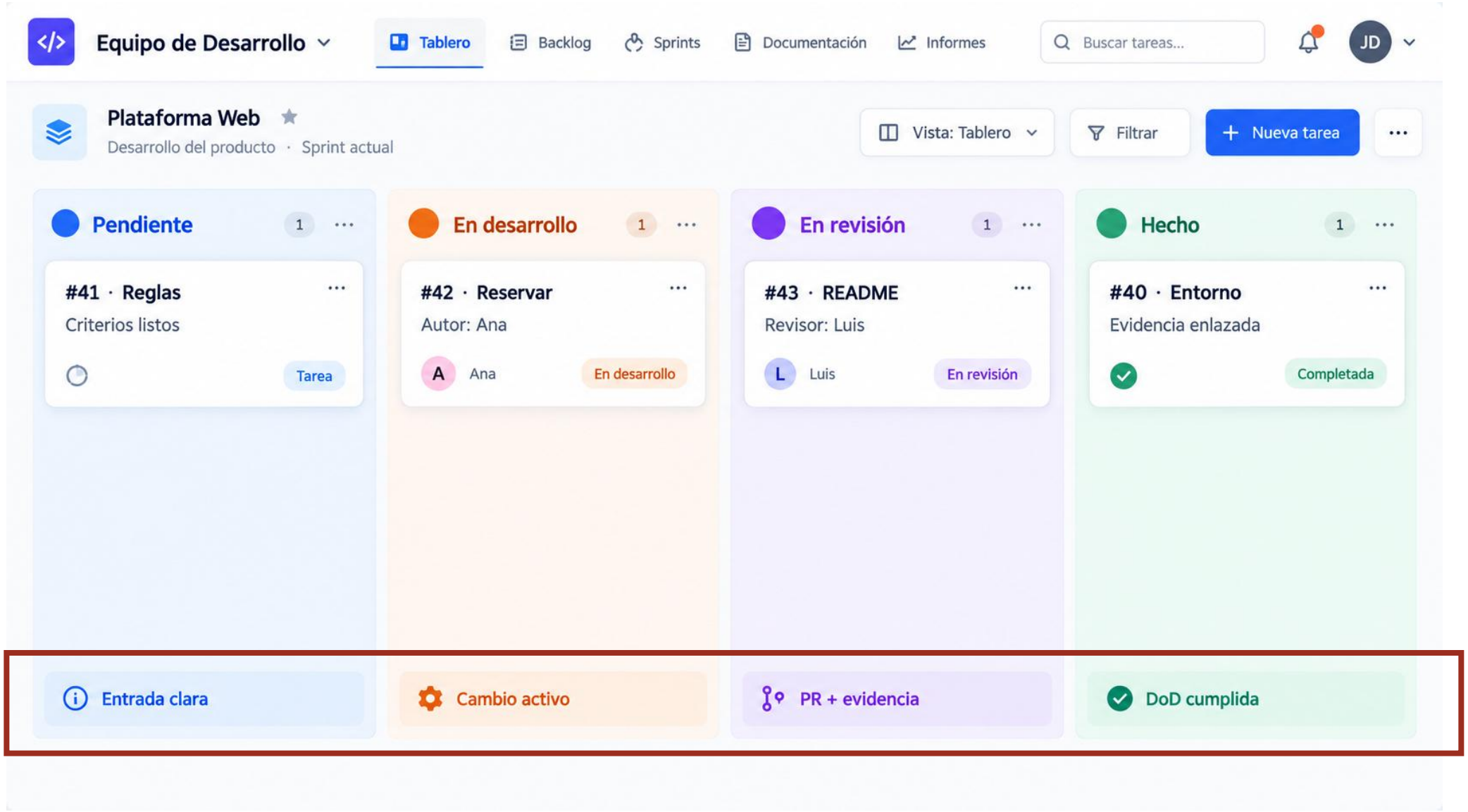
Evidencia: PR enlazado al issue + comentario y respuesta.

Gestión visual

La revisión existe.
¿Cómo sabe el equipo qué está esperando?

Tablero, estados del flujo y Definition of Done.

Tablero y estados: una propuesta para el equipo



Bloqueo: tarjeta visible, causa explícita y una persona que ayude a resolverlo.


ToDo
Doing
Done
ACCEPT

Definition of Done: un acuerdo comprobable

Criterios de aceptación

Para esta reserva:

- si el horario ya está ocupado,el sistema rechaza la solicitud.

Definición de Terminado

- Pruebas pertinentes aprobadas.
- Revisión humana resuelta.
- Documentación actualizada.
- Issue, PR y commit enlazados.
- Integrado en la rama acordada.

«Hecho» significa evidencia verificable de calidad compartida.

Scrum Guide, 2020

Ejercicio · Tablero y reglas de avance

Representen el trabajo real de su equipo.

1. Creen 4 estados y añadan al menos 3 issues.
2. Escriban la condición para entrar en revisión y pasar a Hecho.
3. Registren una DoD de 5 criterios verificables.

Evidencia: URL del tablero + docs/definition-of-done.md.

IA en repositorios

La IA puede explicar y proponer.
El equipo debe poder comprobar.

Asistencia para entender, documentar y revisar cambios.

Explicación de código asistida por IA

Pedir contexto verificable

«Explica esta función en este commit.
Señala entradas, salidas y efectos.
Cita líneas y distingue hechos de supuestos».

Comprobar lo explicado

Abrir los archivos citados.
Seguir una entrada concreta.
Ejecutar un caso límite.
Corregir lo que no se sostiene.

Si falta contexto, la respuesta debe reconocerlo y pedirlo.

IA: mensajes de commit y resúmenes de PR

Commit

Entrada: diff preparado.
Salida: intención breve.

«Propón un mensaje según nuestra convención, solo para este diff».

Pull Request

Entrada: diff + issue + resultados.
Salida: qué, por qué y verificación.

«Resume y distingue pruebas ejecutadas de pendientes».

La persona autora contrasta cada afirmación antes de publicar.

Code Review asistido y validación humana

IA señala un posible fallo → **Persona lo reproduce** → **Equipo acepta, corrige o descarta** → **Pruebas y revisión**

Hallazgo útil Archivo + línea + escenario + consecuencia.

Decisión trazable Motivo, cambio aplicado y evidencia de validación.

Una recomendación convincente puede ser incorrecta. La aprobación es humana.

Ejercicio · ¿Aceptarían esta recomendación de IA?

La IA sugiere eliminar una validación porque «el front-end ya la hace».

1. Identifiquen qué supuesto hace la recomendación.
2. Diseñen una comprobación que lo confirme o lo refute.
3. Registren aceptar, modificar o rechazar, con una razón.

Evidencia: decisión en el PR; si no usan IA, registren «sin uso».

Entregable · Configuración del entorno colaborativo

Repositorio	README, estructura, inventario y reglas de cambio.
Tablero	Issues reales, responsables, estados y políticas.
Workflow	Ramas, commits, PR, revisión e integración.
DoD	Criterios verificables y evidencia de cumplimiento.

Entrega: un enlace principal desde el que se pueda recorrer toda la evidencia.

Workflow propuesto: una tarea hasta «Hecho»

Issue con criterios → **Rama corta + commits** → **PR + pruebas + revisión** → **Integración + DoD**

AUTOR

Explica y verifica.

REVISOR

Contrasta el cambio.

EQUIPO

Resuelve el impacto.

INTEGRADOR

Comprueba acuerdos.

Cada entrega: versión identificada, línea base y evidencia conservada.

Taller · Configurar y demostrar el entorno

Trabajo del equipo	Resultado visible
Integrar inventario y acuerdos	README como punto de entrada
Completar tablero, workflow y DoD	Políticas y responsables explícitos
Recorrer un cambio pequeño	Issue → rama → PR → revisión
Verificar enlaces y reconstrucción	Versión identificada y demo lista

El equipo debe mostrar qué adaptó, por qué y cómo se verifica.

Revisión entre equipos · Evidencia, no promesas

Criterio	Lo que el otro equipo debe poder comprobar
Repositorio	Clonar, orientarse e identificar la versión
Tablero	Relacionar estados con el trabajo y los bloqueos
Workflow	Seguir un cambio y encontrar su revisión
DoD	Reconocer evidencias de cada criterio acordado

Retroalimentación: una fortaleza y una mejora concreta antes de entregar.

Comprobación · ¿Qué falta para confiar?

Versiones	Guardé el archivo. ¿Ya existe un commit?
Gestión visual	Cerré el PR. ¿Puedo mover la tarjeta a Hecho?
Configuración	Creé un tag. ¿Ya aprobé una línea base?
IA	La IA dice que pasa los tests. ¿Qué evidencia pido?

Respondan con una condición comprobable, no solo con «sí» o «no».

Cierre · Un cambio que se puede explicar

Qué cambió.
Por qué cambió.
Quién lo revisó.
Cómo se comprobó.

Repositorio + tablero + workflow + DoD

Fuentes

NASA	Systems Engineering Handbook · 6.5 Configuration Management nasa.gov/reference/6-5-configuration-management/
Chacon y Straub	Pro Git · Capítulos 2 y 3 git-scm.com/book/en/v2
GitHub Docs	GitHub flow · About Projects docs.github.com
Conventional Commits	Especificación 1.0.0 conventionalcommits.org/en/v1.0.0/

Fuentes

Schwaber y Sutherland	The Scrum Guide (2020) · Definition of Done scrumguides.org/scrum-guide.html
GitHub Docs	Application card: GitHub Copilot Agents docs.github.com/en/copilot/responsible-use/agents
pytest · reporte	Issue #13033 · 6 de diciembre de 2024 github.com/pytest-dev/pytest/issues/13033
pytest · corrección	PR #13034 · conversación, revisión y diff github.com/pytest-dev/pytest/pull/13034